İlerleme raporu: case arayışından geri besleme kısıtlı sıralı hesaplamaya

Kapsam. 27–28 Ekim 2026 arasında yapılanlar. 29 Ekim'de bağımsız bir denetimden geçti ve düzeltildi (§2.8). Başlangıç noktası, hocanın "bu proje hangi problemi çözüyor?" sorusu ve bunun sonucunda araştırmanın "modeli FPGA'ye sığdırmak" yerine "modeli FPGA'nin fiziksel kısıtlarından türetmek" yönüne dönmesi.

İlgili belgeler:

- `docs/08_case_arastirmasi.pdf`: case araştırması;
- `docs/09_faz0.pdf`: Faz 0 sonuçları;
- `docs/10_teori.pdf`: teori çerçevesi.

Kod ve sonuçlar: `sw/fbseq/`, `model/fbseq/*.json`, `scripts/`.

---

1. Tek bakışta

	Durum
Hocanın case sorusu	**Cevaplandı:** deterministik dil modeliyle kayıpsız telemetri ve log sıkıştırma; yan case olarak aynı modelle anomali tespiti
Yeni araştırma yönü	**Belirlendi:** geri besleme kısıtlı (feedback-constrained) sıralı hesaplama. Döngüdeki mantık derinliği tasarımın merkez değişkeni
Faz 0 (fizibilite)	**Büyük ölçüde tamam:** 4 hipotezden ikisi tuttu, biri kısmen tuttu, biri Faz 1'e kaldı
İlk teori sonucu	**Var:** A5 için tek LUT seviyesinde en az state, sıkı biçimde 6 bit (SAT tanığı ve bilgi alt sınırı). Birleşik yapı (blok core + fiber), S_4, A_5, PSL(2,7) ve A_6'da alt sınırı SAT'sız ve tek LUT6 seviyesinde veriyor; PSL(2,8)'de vermiyor. FPGA'de fan-in 6'yı aşmak Fmax'i 1,5–2,9 kat düşürüyor
Gerçek veride kalite	Boolean model, doğru eğitimle pencere modelini geçiyor. 100 KB sözlüklü zstd'ye göre eşit koşullarda %7–14 daha az bit (GRU %15–21). Önceki %25 / %32, ideal LM kod uzunluğunu zstd'nin başlıklı dosya boyutuyla karşılaştırıyordu
Literatürdeki konum	Üç hatlı tarama yapıldı; rakipler ve boşluklar belirlendi
Son biten	Transition codebook: olumsuz. 2-bit / 3-bit bloklar Loghub'da set/keep'in gerisinde (1,680 / 2,237). Gumbel-ST birebir yeniden üretildi

---

2. Kronoloji

2.1 Hız sorusunun cevabı: CPU baseline

- Karpathy'nin `run.c`'si, stories260K'yı kartla aynı koşulda çalıştırdı: Apple M1, tek thread, 256 token, greedy. `make cpu-baseline`; `model/zoo/cpu_baseline.json`.
- Sonuç:

Platform	token/s	Token başına cycle
M1, -Ofast	14 397	~222 bin
M1, -O3	6 718	–
FPGA, 25 MHz	496	50 397

CPU mutlak hızda 29 kat önde; token başına cycle'da FPGA 4,4 kat verimli.

- Sunuma eklendi: slayt 2 ("dört soru, dört cevap") ve yeni slayt 46 (kaybın kaynağı ve CPU karşılaştırması). Commit `bee67be`.

2.2 Case araştırması (`docs/08`, commit `9f6939a`)

Dört paralel araştırma (sıkıştırma, CAN bus, sertifikasyon, rakip platformlar) ve bir puanlama matrisi:

- **Ana case: kayıpsız sıkıştırma.** Arithmetic coding'de encoder ve decoder olasılığı bit bit aynı hesaplamak zorunda. Float modeller bunu farklı donanımlarda garanti edemiyor (Ballé 2019, llama-zip, NNCP v2). Bizim bit-exact integer engine'imiz bunu yapısı gereği sağlıyor.
- **Yan case: anomali tespiti.** –log p hem kodlama maliyeti hem anomali skoru olduğu için aynı model iki işi birden yapıyor (telemetri ya da CAN bus).
- **Çapraz gereksinim: sertifikalanabilirlik.** EASA AI rehberi FPGA'yi mevcut sertifikasyon yolunun içinde sayıyor, GPU'yu kapsam dışında bırakıyor.
- **Dürüst konumlandırma.** Ham hızda ve enerjide üstünlük iddia edemiyoruz. MCU'lara göre 5–26 kat hızlıyız.

2.3 Yön tartışması

Hocanın yönü, iki ayrı yorum metni üzerinden tartışıldı. Kendi ölçümlerimiz yönü belirledi:

- **BRAM darboğaz:** stories260K'da %72; iki model RAMB18 parçalanması yüzünden place edilemedi.
- **Logic kaynakları boş:** LUT bel'lerinin en fazla %31'i kullanılıyor, DSP hiç kullanılmıyor.
- **Kritik yol:** gecikmenin %73'ü routing.

Bunun sonucunda:

- **Memory-centric model elendi**, çünkü BRAM zaten dolu.
- **Logic-native yön seçildi:** boş duran logic kaynağını kullanmak.
- **"Neden recurrent?" sorusunun cevabı:** belirleyici olan recurrence değil, loop-carried dependency'nin derinliği. Döngüsü bir mux kadar olan, geri kalanı pipeline'lanan bir yapı hedeflendi. Encoder tarafında girdinin tamamı baştan bilindiği için paralellik mümkün.

- **Dört aday:**
- A: pencereli feedforward;
- B: input-gated Boolean recurrence (set/keep, toggle, 2- ve 3-bitlik bloklar);
- C: GRU;
- D: microgpt.
- **Teorinin önerdiği tasarım:** transition codebook. Her blok, keyfi bir tablo yerine m öğrenilmiş dönüşümden birini seçiyor; döngüde her bit için tek bir LUT(kb + $\log_2$m) kalıyor.

2.4 Faz 0 altyapısı (commit `985fbba`)

- **`sw/fbseq/fbseq.py` (CPU):** dört aday, cebirsel görevler (flag, parity, mod 3, A5), Loghub byte LM, klasik sıkıştırıcı baseline'ları (gzip, xz, zstd, zstd + sözlük).
- **`sw/fbseq/fbseq_gpu.py`:** aynı modeller, B için associative scan, cuDNN GRU, paralel işler. Mac'te test edildi.
- **`scripts/colab_faz0.sh`:** Colab'da tek komutluk çalıştırma. Kullanıcının hesabına A100 yerine T4 düştü. Küçük modellerde T4, M1'den belirgin hızlı değil; tek istisna GRU (~2 kat).
- **Colab CLI** (`google-colab-cli 0.7.5`) kuruldu ve giriş yapıldı.
- Tarayıcıdaki runtime'a bağlanamıyor; CLI yalnız kendi açtığı runtime'ları yönetebiliyor.
- Hesapta 0 compute unit var; A100 açılmadı.
- **`sw/fbseq/loop_bench.py`:** döngü devrelerini Yosys + nextpnr ile place edip Fmax ölçüyor.

2.5 Faz 0 deneyleri

Ayrıntı `docs/09_faz0.md`'de; özet:

1. **Döngü derinliği** (`loop.json`; 29 Ekim'de düzeltilmiş düzenekle): tek mux 668, 4:1 mux 694, 8:1 mux 486 MHz. State'e bağlı 1/2/3/4 seviye: 378 / 238 / 175 / 151 MHz. İlk ölçümde tek seviyeli döngülerin sınırını çıkış XOR ağacı koyuyordu (§2.10). **H1 tuttu.**
2. **Cebirsel görevler** (`synth.json`, 3 seed): set/keep → flag; + toggle → parity; 2 bit → mod 3; 3 bit → A5'te 0,32. Sıralama teoriyle bire bir. **H2 tuttu.**
3. **Byte LM** (`bytes.json`, `base.json`, Loghub, 256 B bağımsız paket):

Model	bpb
GRU	**1,294**
Pencere 8	1,453
B · set/keep (soft→hard)	1,583
zstd + 100 KB sözlük	1,90
zstd, sözlüksüz	5,51
microgpt	1,917 (bu protokolde yetersiz eğitim)

1. **Soft–hard farkı** (`gap.json`):
2. B'nin soft hali 1,398; pencere modelini geçiyor.
3. Aynı ağırlıkla hard 2,77; uyarlanmış hard 1,583. Uyarlanmış hard'ın soft referansa açığı 0,185. Bu bir performans açığı; aynı parametrede discretization farkı soft eğitimden sonra +1,37, uyarlamadan sonra −0,23.

4. Soft state'ler 0 ve 1'den uzak.
5. D(t), farklı bit oranı: %6 ile %12 arasında; t ≈ 8–16'da doyuyor.
6. **GRU bellek ufku** (`gru_horizon.json`): 128 byte'ta sıfırlama 1,39, 64'te 1,58. 64–128 byte ötesindeki bilgi işe yarıyor; ölçümde karışık etkenler var (§5).
7. **Hard-aware eğitim** (`hardaware_*.json`):
8. **Loghub:** baştan Boolean forward ve eğim tavlamasıyla **1,418** (pencere 8: 1,453; soft referans: 1,398; iki seed). Sade Hard-ST 1,545, Gumbel-ST 1,685 (ilk koşu arşivi `model/fbseq/archive/`; düzeltilmiş kodla birebir yeniden üretildi). 2-bit ve 3-bit bloklar 1,680 ve 2,237 (1 seed; eşit bütçede gate ağı daralıyor, etkiler ayrılmadı). **H3 kısmen tuttu.**
9. **A5:** softhard 4 koşunun 2'si, test edilen 64 ve 256 uzunlukta %100; baştan hard 8 koşunun hiçbiri 0,13'ü geçmiyor.
10. **Ödünleşim (denenen reçetelerle):** dil modelinde baştan Boolean en iyi; A5'te başarılı olan yalnız softhard. Başka reçetelerin başarısız olacağı gösterilmedi.

2.6 Teori (`docs/10_teori.md`; commit'ler `1c348ab`, `f45da5e`)

- **Biçimsel model:** K-LUT recurrent devre, block-local yapı, maliyet dörtlüsü (S, D, L, R).
- **Bilinen ve atıflı:** cascade'in iteration bound'u (Leiserson–Saxe, Thm 11), grup word problem'leri için encoding'ler, Myhill–Nerode alt sınırı.
- **Bizim sonuçlarımız (A5 küçük vakası):**

Gerçekleştirme	State	Döngü	Fmax (MHz)
Öğrenilen, tam tablolu 3-bit bloklar	24	2 seviye	486
Elle, ters kod r = p^{-1}	15	LUT5	688
Bağımsız dense bloklarla en küçük, w = 2 (V4 + A4 coset'leri; K = 5'te 9 bit; K = 4'te imkânsız)	7	LUT6	614
SAT, alt sınırda	**6**	LUT6	643

Fmax değerleri düzeltilmiş düzenekle (§2.10); ilk ölçümler 399–458 MHz'di ve döngüyü değil çıkış XOR ağacını ölçüyordu.

İspat yolu: Hartmanis–Stearns SP partition'ları, grup makinelerinde alt grupların coset'lerine karşılık gelir; injectivity ⇔ alt grupların kesişimi birim eleman; A5'in 59 alt grubu tarandı. Kod: `sw/fbseq/theory_sat.py`.

Fmax değerleri yalnız state döngüsünün, iki seed ortalaması. 6, 7 ve 15 bit aynı hız ölçeğinde; iki seed bir sıralama vermiyor.

- **Kararlılık:**
- hata rekürsiyonu sınırı ve koşullu beklenen değer sınırı;
- tek bit için $\mathbb{F}_2$ tam eşitliği;
- bloklara genelleme: birleşen map'ler farkı siler, permütasyonlar durumları ayrı tutar (aynı sonraki kontroller şartıyla);
- kayba bağlantı (Lipschitz sınırı);

- bellek ile dayanıklılığın bağı yalnız bağımsız 1-bit keep/toggle için geçerli. Genel bloklar hata düzeltebilir (majority karşı örneği); 60 durumu tek hatadan düzelten bir kod için en az 10 bit gerekir. Önceki sürüm bunu genelliyordu (§2.8).
- **Sanı** (literatür ajanının taslağı, kontrol edilmeli): k-bitlik bloklar tam olarak, sözdizimsel monoidinin basit grup bölenleri S_{2^k}'yi bölen dilleri tanır. Test edilebilir öngörüleri var.

2.7 Literatür taraması (üç hat)

Klasik:

- Leiserson–Saxe 1991; Lin–Messerschmitt 1991 (FSM look-ahead);
- Stearns–Hartmanis 1961 (partition pair: Teorem C'nin çekirdeği);
- Derby 2001 (CRC state-space dönüşümü: doğrusal durum; doğrusal olmayan genellemesi bulunamadı);
- Barkalov 2021 (LUT tabanlı FSM);
- Shende 2003 (tersinir devre paritesi; önceki uyarlamamız yanlıştı, §2.8).

İfade gücü: Sarrof 2024, Grazzi 2025, Merrill 2024, PD-SSM 2025 (en yakın nöral akraba), Knorozova & Ronca 2024, Zhang & Li 2026 (coset aşaması; bizim başarısız koşularımız için yalnız bir hipotez).

Logic ağları:

- RDDLGN (EdgeFM 2025 atölyesi): keep yolu yok, donanım yok.
- Gating LGN (NeurIPS 2026 AXIOM atölyesi): bit başına keep/write mux, donanım yok.
- Mind the Gap 2025, Kim 2026 (Hard-ST ve CAGE), HM-RNN 2017.

Bulunamayanlar (bizim için boşluklar):

- sentezlenmiş bir recurrent logic ya da LUT modeli ve Fmax'i;
- logic veya LUT ağlarıyla byte düzeyinde bpb;
- döngü derinliğini tasarım kısıtı yapan bir çalışma;
- Boolean transition codebook;
- adım bazında discretization gap ölçümü;
- LUT fan-in kısıtlı en küçük grup gerçekleştirmeleri.

2.8 Bağımsız denetim (29 Ekim)

`docs/10` ve bu rapor dışarıdan denetlendi. Denetimin bütün sayısal kontrolleri burada ayrıca yeniden üretildi: `sw/fbseq/witness_check.py` → `model/fbseq/witness_check.json`. Bu script z3 kullanmıyor ve `theory_sat.py` ile kod paylaşmıyor.

Doğrulanan çekirdek:

- 6 bitlik SAT tanığı: 60 farklı kod, 1 080 geçiş koşulunun hepsi.
- 59 alt grup.
- Bağımsız dense bloklarla K = 4 imkânsız, K = 5'te 9 bit, K = 6'da 7 bit.
- Slope sonucu: 1,4186 / 1,4176.

Düzeltilenler:

Konu	Önceki ifade	Düzeltme
Tersinir devre paritesi	"Bir biti hiç değiştirmeyen devre yalnız çift permütasyon üretir"	Yanlış; CNOT karşı örneği. Shende Lemma 11'in koşulu daha güçlü
Sağa çarpma, positional kod	"Mux ile güncellenir"	Kısmi base'de yerel değil; ters kod r = p^{-1} yerel
β_K	"En küçük state"	Yalnız sabit alfabe kodlaması, harften gelen kontrol ve bağımsız dense bloklar için
Δ_hard = 0,185 → 0,02	Discretization farkı	Performans açığı (farklı parametre noktaları). Aynı parametrede fark: +1,37 / −0,23
A5 seed'leri	"Yarısı tam uzunluk genellemesi; diğerleri 1/3'te coset aşamasında"	2/4, yalnız 64 ve 256 uzunlukta; başarısızlar 0,36 → 0,21 ve 0,25 → 0,09; coset aşaması hipotez
"Soft aşama şart"	Zorunluluk	Denenen reçetelerde softhard 2/4, hard-forward 0/8
Bellek ile dayanıklılık	Genel	Yalnız bağımsız 1-bit keep/toggle; majority karşı örneği, 60 durum için ≥ 10 bit
A2	"Her çevrim tek LUT ve tek register"	Her çevrimde #LUT = #register (tek seviyeli F)
Fiziksel Pareto	"6 bit Pareto ucunda"	6 bit, iki yerleştirmede ~455 MHz; 7 ve 15 bitle aynı hız ölçeğinde
Gumbel 1,685	Kaynaksız	İlk koşu arşivi `model/fbseq/archive/`; düzeltilmiş kodla birebir yeniden üretildi
%25 daha az bit	İdeal LM ile başlıklı zstd	Eşit koşullarda %7–14 (aşağıda)

Denetimin ötesinde bizim bulduklarımız:

- **SAT tanığının yapısı genellendi.** Tanık, 15 durumlu bir coset core (V4'ün coset'leri) ve iki koşullu toggle'dan oluşuyor; bunu denetim fark etti. Biz bunu bir lemmaya çevirdik: $H \cong Z_2^k$ için tek seviyede $S = \lceil \log_2 |G| \rceil$ bit, yani her zaman alt sınır; fan-in ≤ S − k + 1 + w. Transversal seçimi $\mathbb{F}_2$ üzerinde lineer bir sistem; çözülünce fan-in S + w − k'ya iniyor. A5'te 300 etiketlemenin 300'ü fan-in 6 verdi; SAT'ın 6 344 saniyesine karşı milisaniyeler. PSL(2,7) 8 bit / fan-in 8, A6 9 / 9. S_4'te 5 bit / fan-in 4'e kadar iniyor (`core_fiber.py`). İlk sürümdeki "K = 6 yalnız özel etiketlemelerle" ifadesi yanlıştı: nadir olan rastgele transversal'in doğru çıkmasıydı.
- **zstd başlığı.** zstd her 256 byte'lık pakete 10 byte frame başlığı yazıyor; bu ~0,31 bpb ediyor. LM'ye coder sonlandırması eklenip zstd'den başlığın çıkarılabilir kısmı düşüldüğünde avantaj B için %7–14, GRU için %15–21. Ayrıntı `docs/09` §4.
- **Checkpoint probe'larında iki ölçüm hatası.** Head'i yeniden eğitme, eğitimdeki reçeteyi kullanmıyordu. Gecikmeli okumada "yakın byte'lardan zaten tahmin edilebilir" kontrolü yoktu.

İkisi düzeltildi ve yeniden koşuldu (`docs/10` §8).

2.9 Teori: birleşik yapı ve beş grupta FPGA ölçümü (29 Ekim akşam)

`docs/10` §5.3–5.4; kod `sw/fbseq/core_fiber.py`, `realize.py`, `loop_bench.py` (gc: türü).

- **Birleşik yapı.** Core, $H \cong Z_2^k$'yi içeren alt grupların coset bloklarından kuruluyor ($\cap O_i = H$). Fiber, Z_2^k toggle'larından oluşuyor ve gauge'u $\mathbb{F}_2$ üzerinde lineer çözülüyor. Doğruluğu ispatlı. Bağımsız bloklar ve tek core + fiber bu yapının özel durumları.
- **Bulunan en küçük state** (her kod kaba kuvvetle ve Verilog simülasyonuyla doğrulandı):

Grup	Alt sınır	K = 5	K = 6
S_4	5	5	5
A_5	6	7 (önce 9)	6
PSL(2,7)	8	**8**	8
A_6	9	12	**9** (önce 10)
PSL(2,8)	9	–	12 (9 bit ancak K = 8'de)

Beş grubun dördünde bilgi alt sınırı tek LUT6 seviyesinde gerçekleşiyor. PSL(2,8)'de gerçekleşmiyor: indeksi ≤ 16 olan alt grupları yalnız nokta stabilizer'ları ve bunların kesişimleri Z_2^k değil. A5'te K = 5 ile 6 bitin mümkün olup olmadığını SAT ile arıyoruz. - **FPGA (XC7A100T, ~120 bit, iki seed; düzeltilmiş düzenek, §2.10).** Aynı state'te fan-in 6'yı aşmanın ciddi bir fiziksel bedeli var: - A6, 9 bitte: fan-in 11'de 156 MHz ve 6 232 LUT; fan-in 6'da 636 MHz ve 142 LUT. - PSL(2,7), 8 bitte: fan-in 10'da 193 MHz; fan-in 5'te 669 MHz. - K ≤ 6 olan tasarımların hepsinde kritik yolda tek LUT var ve Fmax 599–669 MHz; kodlar arasında ayırt edilebilir bir fark yok. İlk ölçümde gördüğüm "positional daha hızlı" farkı düzeneğin artefaktıydı. - PSL(2,8)'de en az state'li tasarım en yavaşı: 9 bit / fan-in 8 317 MHz, 12 bit / fan-in 6 641 MHz. - Sonuç: teorinin donanımdaki karşılığı önce LUT6 eşiği. Eşiğin altında state biti sayısı Fmax'i değil, LUT ve register sayısını etkiliyor.

2.10 İkinci denetim: karşı örnekler, yapıcı teorem, ölçüm düzeltmesi (29 Ekim gece)

Denetim çıktıları burada bağımsız olarak yeniden üretildi; ayrıntı `docs/10` §5.3–5.4 ve §8.

Düzeltmeler (hepsi doğru, hepsi uygulandı):

- **"Her grupta fan-in S + w − r_2" sanısı yanlış.**
- Z_2'de formül 1 diyor, gerçek 2.
- Z_8'de 5 040 kodlamanın hepsinde fan-in 4, formül 3 diyor; burada ayrıca tarandı.
- Kendi tablomuzdaki Z_2^3 satırı bunu zaten gösteriyordu.
- **Gauge her etiketlemede bir core bitini düşüremez.**
- A5'te sertifikalı bir karşı örnek var: yalnız (1,1) fonksiyoneli uygulanabilir. `gauge()` ile doğrulandı.
- 300/300 gözlemi geçerli, ama "her etiketlemede" geçerli değil.
- **Kontrol girişleri.**
- Tanım 1 her çıkışın kontrol bitlerini seçmesine izin veriyor; fan-in sayımım ve SAT sorgum ise iki kontrol bitini her çıkışa zorunlu yazıyordu.

- Raporlanan 31 kodun en büyük fan-in'i, kontrol seçilebilir sayılınca da değişmedi.
- SAT sorgusu genelleştirildi (`theory_sat.py --coding`). Sağlaması: tek mertebeli inşanın kodu yeni modelde K = 6 ile SAT, eski modelde UNSAT.
- **kb = 1'de hatanın büyümemesi önermesi daraltıldı.** Döngü hatayı çarparak büyütmüyor, ama dış terimler birikebilir. kb > 1 bloklarda Hamming uzaklığı büyüyebilir.
- **FPGA ölçüm düzeneği.** Kritik yol analizi gösterdi ki tek seviyeli bütün döngülerde sınırı çıkış XOR ağacı (2–3 LUT) koyuyordu. Düzenek düzeltildi ve bütün ölçümler yeniden koşuldu.
- Faz 0'daki yerel tek seviyeli döngüler: 441–516 → 667–708 MHz.
- A5 gerçekleştirmeleri: 424–473 → 589–724 MHz.
- dep1–dep4 değişmedi; onlar zaten döngüyü ölçüyordu.

Yeni teorem (denetimden; ispatı burada kontrol edildi).

- Varsayımlar: $G = \langle a, b \rangle$, $|a|$ tek, $H \cong Z_2^k$.
- Sonuç: her core etiketlemesinde $S = \lceil \log_2 |G| \rceil$ ve $K \le S + 2 - k$.
- Fikir: a'nın coset yörüngeleri tam $|a|$ uzunlukta. Transversal a ile çarpmaya kapalı seçilince fiber yalnız b'de değişiyor, ve bir kontrol bitini okumak yetiyor.
- Guralnick–Kantor (2000) ile bu, uygun alfabede her sonlu abelyen olmayan basit gruba uzanıyor: $K \le S + 2 - r_2(G)$.
- Kontrol: 81 etiketlemede fan-in tam olarak sınıra eşit çıktı (A5 6, PSL(2,7) 8, A6 9, PSL(2,8) 8). Gauge karşı örneğinde gauge K = 7'de kalıyor, inşa K = 6 veriyor.

Grup tasarımları yeniden ölçüldü (§2.9'daki sayılar güncel). K ≤ 6 tasarımları 420–540'tan 599–669 MHz'e çıktı; K > 6 tasarımlarının sınırı zaten döngüydü, değerleri az değişti.

Öğrenme: doğru dönüşümleri bulmak mı, seçmek mi? Denetimin önerdiği ayrıştırma (`docs/10` §7):

- Dönüşümler inşadan sabit, yalnız seçim ağı ve head öğreniyor:
- yalnız 3 doğru dönüşüm: softhard 4/4;
- yanlış girdiler eklenince 0/8;
- aynı şey, kısa dizilerden başlayan curriculum'la: 4/8.
- Dönüşümler de öğreniliyor: curriculum işe yaramıyor (codebook 0/8 ≤ 0,03; B_block3 softhard 0/4, slope 1/4).
- Sonuç: seçim kısmı bir eğitim reçetesi sorunu, ve curriculum büyük ölçüde çözüyor. Zor olan geçiş tablolarını bulmak. Bu, yapıcı inşaların pratik değerini artırıyor: dönüşümler teoriden, seçim ve head öğrenmeden.

---

3. Bugün ne iddia edebiliriz, neyi edemeyiz?

Edebiliriz:

1. Döngüdeki mantık derinliği ile Fmax arasındaki ilişkiyi XC7A100T'de ölçtük.
2. Boolean blok recurrence'ın ifade gücü sıralaması teoriyle örtüşüyor; ölçtük.
3. A5 için tek seviyeli fan-in 6 gerçekleştirmenin en küçük state'i tam olarak 6 bit. Bağımsız dense bloklarla (kontrol yalnız harften, w = 2) 7 bit. Hepsi bağımsız olarak doğrulanmış ve place edilmiş. 6 bitlik çözümün yapısı (coset core + fiber) bir lemmaya bağlandı.

4. Gerçek log verisinde Boolean recurrence, doğru eğitimle aynı bütçedeki pencere modelini ve sözlüklü zstd'yi geçiyor. Eşit koşullarda zstd'ye göre avantaj %7–14. Kapsam: iki seed, ideal bit maliyeti artı tahmini coder maliyeti, aynı dosyaların son %20'si; model float.
5. Soft–hard farkı ve adım bazında state ayrışması ölçüldü. Hard-forward ve eğim tavlaması, hard modelin soft referansa göre performans açığını 0,185'ten 0,02 bpb'ye indiriyor.

Edemeyiz (henüz):

1. Gate ağları ve olasılık head'i float. Model bütünüyle bir LUT devresi değil (H4, Faz 1).
2. Gerçek sıkıştırma yok: range coder ve bit-exact encode/decode yapılmadı.
3. Görülmemiş log sistemlerine genelleme test edilmedi.
4. Bütün donanım sonuçları simülasyon ve STA'ya dayanıyor; enerji ölçülmedi.
5. A5'in eğitimle güvenilir biçimde öğrenilmesi çözülmedi: softhard 2/4, hard-forward 0/8, öğrenilen codebook 0/32. Doğru dönüşümler verilince seçim curriculum'la öğreniliyor (4/8).
6. Teoride yeni olan yalnız A5 küçük vakası; genel teorem (Problem C) açık.

---

4. Q1 açısından değerlendirme

- **Yeni ve savunulabilir bir çekirdek oluştu:** ifade gücü, encoding, döngü maliyeti ve fiziksel maliyet arasındaki ilişki; ölçülmüş FPGA karşılığı ve gerçek bir uygulama (sıkıştırma) ile birlikte.
- **Henüz eksik:**
- genel bir teorem (A5'in ötesi);
- Boolean devreleri güvenilir biçimde eğiten bir yöntem;
- LUT'a çevrilmiş tam model.
- **Değerlendirme:** Bugün sağlam bir bitirme ve iyi bir konferans çalışması düzeyindeyiz. Q1 bir eşik değil, hakeme bağlı bir hedef. Bu eksikler kapandıkça argüman güçlenir; kaçının yeteceğini önceden söyleyemeyiz.

---

5. Hatalar, düzeltmeler ve sınırlar (şeffaflık için)

Konu	Ne oldu	Durum
"A5 için en az iki LUT seviyesi gerekir" iddiası	Yanlıştı; A5 tek seviyede (LUT5) gerçekleştirilebiliyor	Düzeltildi
"Büyük basit grup → derin döngü" alt sınır sanısı	Genel olarak yanlış; one-hot encoding ile her basit grup fan-in 5'te	Karşı örnek belgelendi
"B'nin geride kalmasının sebebi mimari değil"	Erken bir iddiaydı; soft model aynı fonksiyon sınıfında değil	Düzeltildi
hardst yöntemi	Yanlışlıkla eğim tavlaması kullanıyordu	Düzeltildi, yeniden koşuldu

Konu	Ne oldu	Durum
kb ≥ 2 bloklarda NaN	Bit bit multilinear relaxation'ın Jacobian'ı 1'den büyük; gradyan patladı	Kararlı dağılım formuna geçildi; CPU ve GPU soft ve hard eşit
microgpt 1,917	Aynı protokol bir transformer için yetersiz	Adil eğitim yapılacak
GRU bellek ufku	Eğitim/değerlendirme uyumsuzluğu; blok sıfırlama ≠ kayan pencere	Kayan pencere GRU'su yapılacak
T4 hızlanması	Küçük modellerde beklenen hızlanma yok	Kayıt altında
Araştırma ajanı	Kullanıcının derlediği `run.c` binary'lerini kendiliğinden çalıştırdı; repoya yazmadı	Kullanıcıya bildirildi
PDF oluşturucu	Chrome 156 PDF'i yazdıktan sonra hata koduyla çıkıyor	Dosyanın varlığı kontrol ediliyor
Veri	Loghub küçük; test, aynı dosyaların son %20'si	Leave-one-system-out yapılacak
Bağımsız denetim (29 Ekim)	Teori ve raporda kapsam ve ifade hataları	Düzeltildi; §2.8
zstd karşılaştırması	İdeal LM kod uzunluğu, başlıklı zstd dosya boyutuyla karşılaştırılmıştı	Eşitlendi: %25 → %7–14
Codebook kuyruğu	Bekleme döngüsü `pgrep` ile kendi komut satırını buluyordu, hiç başlamadı	Elle başlatıldı
Probe'lar	Head yeniden eğitimi farklı reçeteyle; gecikmeli okumada kontrol yoktu	Düzeltildi, yeniden koşuldu

6. Çalışan ve sıradaki işler

Bitti:

- transition codebook, olumsuz: A5'te 24 koşunun en iyisi 0,12; Loghub 2,07–2,16 bpb;
- 2-bit ve 3-bit bloklar, Loghub'da eğim tavlamasıyla: 1,680 ve 2,237 (set/keep 1,418);
- Gumbel-ST'nin yeniden üretimi: birebir aynı.

Öncelik sırasıyla sıradakiler:

1. Birleşik yapı için genel koşul: hangi gruplarda bilgi alt sınırı K = 6 ile ulaşılabilir? A5 için K = 5 (SAT sürüyor); daha büyük gruplar.
2. A5'i güvenilir öğrenmek: PD-SSM reçetesi, CAGE, curriculum; amaç seed başarı oranını %50'den yukarı çekmek. Başarısız koşularda iç durumun coset yapısına bakmak.
3. Problem C'nin genelleşmesi:

4. K = 5 ve K = 4 için block-local olmayan SAT;
5. S_4, mod 5, mod 7 görevleriyle sanının testi;
6. β_{K,w} ölçüsünün diğer gruplarda hesaplanması.
7. Kararlılık: tek bit çevirme deneyi ve head'in Lipschitz sabiti.
8. Faz 1:
9. gate ağlarını ve head'i LUT ağlarına çevirmek;
10. bit bit olasılık head'i ve range coder ile gerçek sıkıştırma;
11. leave-one-system-out değerlendirme;
12. adil microgpt eğitimi ve kayan pencere GRU'su.

---

7. Dosya ve commit dizini

Commit	İçerik
`bee67be`	CPU baseline, sunum slayt 2 ve 46
`9f6939a`	Case araştırması (`docs/08`)
`985fbba`	`sw/fbseq` altyapısı, GPU sürümü, Colab script'i, döngü ölçümleri, baseline'lar
`1c348ab`	Teori belgesi, A5 gerçekleştirmeleri, SAT, soft–hard deneyi, kararlı relaxation
`f45da5e`	Faz 0 belgesi, düzeltilmiş hard-aware sonuçları, codebook modeli
`202b576`	Bu rapor
`1ca9f6a`	Eleştirel okumadan düzeltmeler; A5 çözümü öğrenilen sınıfın içinde
`7c61c46`	Bağımsız denetimin düzeltmeleri, `witness_check.py`, coset core + fiber lemması, codebook ve probe sonuçları, zstd başlığı
`03af542`	Bloklar Loghub'da, Gumbel-ST'nin yeniden üretimi
`f1b7ab2`	Gauge'un lineer çözümü; A5'te fan-in 6 SAT'sız
`50fe26b`	Birleşik yapı, dört grupta alt sınır; fan-in'in FPGA'deki bedeli
(bu commit)	PSL(2,8): yapının sınırı ve fiziksel ödünleşim

Dosya	Görev
`sw/fbseq/fbseq.py`	Modeller, görevler, Loghub, baseline'lar (CPU)
`sw/fbseq/fbseq_gpu.py`	GPU sürümü, scan == döngü kontrolü
`sw/fbseq/loop_bench.py`	Döngü devrelerinin Fmax'i
`sw/fbseq/controls.py`	Soft–hard, A5 elle çözüm, maliyet, bellek ufku
`sw/fbseq/hardaware.py`	Hard-forward eğitim yöntemleri

Dosya	Görev
`sw/fbseq/theory_sat.py`	A5 alt grupları, block-local minimum, SAT
`sw/fbseq/codebook.py`	Transition codebook
`sw/fbseq/probes.py`	A5 çözümünü öğrenilen sınıfa yazma; checkpoint probe'ları
`sw/fbseq/witness_check.py`	SAT tanığının ve karşı örneklerin bağımsız denetimi; coset core + fiber
`sw/fbseq/core_fiber.py`	Coset core + fiber, gauge'un lineer çözümü; Z_2^3, Z_8, S_4, A_5, PSL(2,7), A_6; FPGA için kod dışa aktarımı
`sw/fbseq/realize.py`	Birleşik yapı (blok core + fiber): grup ve K başına bulunan en küçük state, doğrulanmış kodlar
`model/fbseq/archive/`	İlk hard-aware koşusunun kayıtları (Gumbel 1,685'in kaynağı)
`scripts/get_loghub.sh`, `scripts/colab_faz0.sh`, `scripts/cpu_baseline.sh`	Veri, Colab, CPU baseline
`model/fbseq/*.json`	Bütün sonuçlar